MOBILE APPLICATION DEVELOPMENT

BSC (HONS) IN INFORMATION TECHNOLOGY

Mobile Application Testing & Debugging

Unit Testing, UI Testing, Emulators vs Real Devices & Debugging Tools



Learning Outcomes

01 Explain why testing and debugging are essential to Android app quality.

02 Distinguish unit tests, UI tests, and instrumented tests.

03 Write basic unit tests using JUnit and Mockito.

04 Write UI tests for screens using Espresso.

05 Compare emulators and real devices for testing.

06 Use Android Studio’s debugging tools: Logcat, Debugger, Layout Inspector, and Profiler.

Why Are Testing & Debugging Required?

- Mobile apps run across many device models, OS versions, and screen sizes.
- Bugs found after release are costly and damage user trust.
- Testing and debugging build confidence that the app behaves correctly before shipping.

Without testing

Bugs surface in production, discovered by real users.

With testing

Bugs are caught early, before release, with repeatable checks.

Design goal

Test early, test often, and at the right layer.

Types of Android Tests: Choosing the Right Layer

Unit Tests

Verify a single function or class, in isolation from the Android framework.

UI Tests

Verify user-facing behavior by simulating taps, scrolls, and input on screens.

Integration Tests

Verify that multiple components work together correctly, e.g. DAO + database.

Manual Tests

Human testers exploring the app; useful, but not repeatable or scalable.

- Key decision: does the check need instant JVM feedback, or a real/emulated Android device?
- For fast feedback during development, unit tests are preferred; UI tests validate the full experience.

Local (Unit) Tests vs Instrumented Tests

Local Tests

Run on the JVM on your machine.

Fast to execute, no device needed.

No access to real Android framework APIs.

Best for pure logic and calculations.

Instrumented Tests

Run on an emulator or a real device.

Access real Android framework and UI.

Closer to real app behavior.

Slower to set up and execute.

Hybrid Approach

Unit tests cover logic.

Instrumented tests cover UI and integration.

Balances speed and confidence.

Matches the shape of the testing pyramid.

What Is Unit Testing?

- A unit test checks that a single method or class produces the expected output for a given input.
- Unit tests run on the JVM, without needing an emulator or a physical device.
- JUnit is the standard framework used to write and run unit tests in Android.

Without unit tests

Bugs in logic surface only during manual testing, or in production.

With unit tests

Logic errors are caught immediately, with fast, repeatable feedback.

Unit tests are appropriate whenever behavior can be checked without the Android UI or system.

Writing a Basic JUnit Test

```
class CalculatorTest {



    @Test
    fun addition_isCorrect() {
        val calculator = Calculator()
        val result = calculator.add(2, 3)
        assertEquals(5, result)
    }

    @Test
    fun divide_byZero_throwsException() {

        assertThrows(ArithmeticException::class.java) {
            calculator.divide(10, 0)
        }
    }
}
```

Common JUnit Assertions

assertEquals

Checks that two values are equal.

assertTrue

Checks that a condition is true.

assertNull

Checks that a value is null.

assertThrows

Checks that an exception is thrown.

```
@Test
fun login_withValidCredentials_succeeds() {
    val result = authManager.login("user@test.com", "pass123")
    assertTrue(result.isSuccess)
    assertEquals("user@test.com", result.user?.email)
}
```

Mocking Dependencies with Mockito

```
val mockRepository = mock(UserRepository::class.java)
`when`(mockRepository.getUser(1))
    .thenReturn(User(1, "Nimal"))

val viewModel = UserViewModel(mockRepository)
viewModel.loadUser(1)



verify(mockRepository).getUser(1)
assertEquals(
    "Nimal", viewModel.userName.value)
```

- Mocks replace real dependencies, such as network or database calls, with controlled fake behavior.
- Mocking isolates the class under test from external systems it depends on.

What Is UI Testing?

- UI tests verify what the user actually sees and interacts with on screen.
- UI tests run on an emulator or a real device, not just the JVM.
- Espresso is Android’s standard framework for writing UI tests.

Find

Locates a view on screen, e.g. by ID or text.

Act

Performs an action, e.g. click, type, or scroll.

Assert

Checks the view’s resulting state.

Writing an Espresso Test

```
@Test
fun login_withValidCredentials_navigatesHome() {
    onView(withId(R.id.emailInput))
        .perform(typeText("user@test.com"))
    onView(withId(R.id.passwordInput))
        .perform(typeText("password123"))
    onView(withId(R.id.loginButton)).perform(click())
```

@Test

Marks the method as a runnable test case.

onView

Locates the target UI element.

matches

Confirms the expected UI state.

Emulators vs Real Devices

Emulator

Runs on your development machine.

Fast to create multiple configurations.

No physical hardware needed.

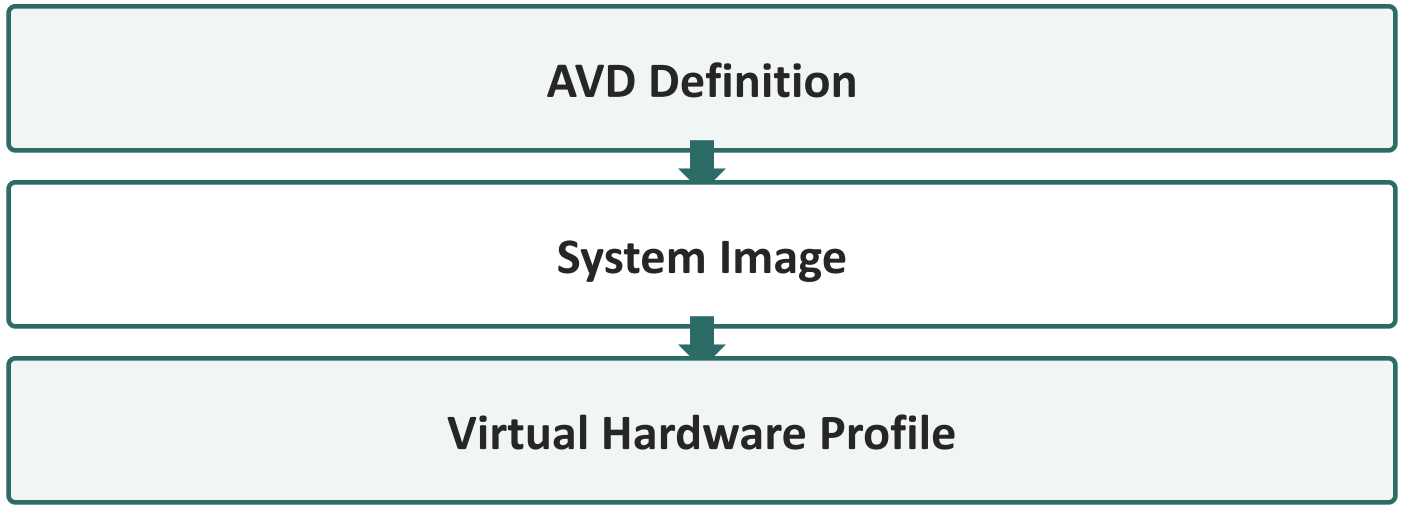
Real Device

Genuine hardware and OS behavior.

Accurate performance and battery testing.

Use emulators for daily development and quick iteration; validate on a real device before release.

Android Virtual Device (AVD) Configuration



API Level
“34”

Screen Size
“6.1-inch”

RAM
“2 GB”

Reading Logcat Output

```
class LoginActivity : AppCompatActivity() {



    private fun login(email: String, password: String) {
        Log.d("LoginActivity", "Attempting login for $email")

        authManager.login(email, password) { result ->
            if (result.isSuccess) {
                Log.i("LoginActivity", "Login succeeded")
            } else {
                Log.e("LoginActivity", "Login failed", result.error)
            }
        }
    }
```

- Use meaningful tags and the correct log level — Verbose, Debug, Info, Warn, Error — to filter Logcat output quickly.

What Is the Android Studio Debugger?

- The debugger lets you pause app execution at a specific line, called a breakpoint.
- While paused, you can inspect variable values, the call stack, and object state.
- Step Over, Step Into, and Step Out control how execution resumes, line by line.
- Conditional breakpoints pause execution only when a specified expression is true.

The debugger works on both emulators and real devices connected via USB.

Core Debugging Tools in Android Studio

Logcat

Streams system and app log messages in real time.

Debugger

Pauses execution to inspect variables and call stacks.

Layout Inspector

Examines the live view hierarchy and layout properties.

Profiler

Measures CPU, memory, and network usage over time.

Typical Debugging Workflow

01 Reproduce the bug

02 Add a breakpoint

03 Run in debug mode

04 Hit breakpoint, pause

05 Inspect variables

06 Form a hypothesis

07 Apply a fix

08 Re-run tests to confirm

Key Takeaways

01 Testing and debugging build confidence that an app behaves correctly before release.

02 Unit tests (JUnit, Mockito) check logic quickly, in isolation, on the JVM.

03 UI tests (Espresso) verify the real user experience on an emulator or device.

04 Emulators support fast iteration; real devices confirm true behavior before release.

05 Logcat, Debugger, Layout Inspector, and Profiler each answer a different question.

06 Debugging is a loop: reproduce, inspect, fix, then re-test.

Discussion / Quick Check

01 What is the difference between a unit test and a UI test?

02 Why do unit tests run faster than UI tests?

03 What role does Mockito play in a unit test?

04 What are the three parts of an Espresso test (onView, perform, check)?

05 Give one reason to test on a real device instead of only an emulator.

06 Name two tools you would use to diagnose a slow screen.

Thank You

Questions & Discussion